

Safeguarding Sensitive Credentials

- Home
- Documentation
- <> API Reference
- Code Samples
- Announcements
- Models
- Release Notes
- FAQ
- GitHub
- Videos

- GET STARTED
- Welcome to SP-API Documentation
- What is the Selling Partner API?
- Selling Partner API Onboarding Overview >

- Terminology
- Frequently Asked Questions >

- CHANGELOG
- SP-API Release Notes
- SP-API Deprecation Schedule

Safeguarding Sensitive Credentials

Learn how to safeguard sensitive credentials for your SP-API Applications.

Protecting sensitive credentials is critical for application security. In the rush to launch a new product, organizations might want to take shortcuts, including hardcoding sensitive credentials. **Hardcoding** is the act of saving credentials outside of a secure credential store, and includes source code, wikis, documentation, spreadsheets, and configuration files. **Credentials** is the term used in this guide to represent passwords, personal identification numbers (PINs), secure shell (SSH) keys, API keys, encryption and decryption keys, and Amazon SP-API client credentials or tokens.

The following is an example of hardcoded credentials:

JSON

```
{ "AssumedRoleUser": {
  "AssumedRoleId": "AROAIEGLQIIQUSJ2I5XRM:s3-access",
  "Arn": "arn:aws:sts:AWS-ACCOUNT-NUMBER:assumed-role/s3-read/s3-access"
},
"Credentials": {
  "SecretAccessKey": "wZJph6PX3sn0ZU4g6yfXdkyXp5m+nwkEtdUHwC3w",
  "SessionToken": "FQoGZXIvYXdzENr/////////REST-OF-TOKEN",
  "Expiration": "2018-11-02T16:46:23Z",
  "AccessKeyId": "ASIAxQZXUENECYQBAAQG"
}
}
```

Solution providers who hardcode credentials create a leading cause of security breaches. Hardcoded credentials expose themselves to attackers through code repositories, documentation, or configuration files.

This guide establishes mandatory security practices for sensitive credentials. To eliminate credential exposure risks, solution providers must implement secure credential storage through encrypted credential stores, automate annual key rotation, and maintain private code repositories with robust access controls and monitoring systems.

DIRECTORIES

SP-API Models

SP-API Seller Use Cases

SP-API Vendor Use Cases

Seller Central URLs

Vendor Central URLs

REGISTRATION

SP-API Registration Overview

Developer Registration Request Status

Third-Party Provider Registration

Service Provider Registration and Role Approval Process Overview

Website Guidelines for Public Developers and Service Providers

User Permissions for Service Providers

Register your Application

Update your Application Information

Rotate your Application's LWA Credentials

View your Application Information and Credentials

Learn How Sellers Authorize Service Providers

Selling Partner API Roles

Policies and Agreements

About Facial Data

AUTHORIZATION

Authorize Applications

Renew Authorizations

Revoke Authorizations

Authorization Limits

Amazon Data Protection Policy (DPP) requirements

To comply with the **Secure Coding Practices** requirements in the DPP, solution providers must:

- Ensure that sensitive credentials, including encryption keys, secret access keys, or passwords, are not hardcoded in their code.
- Ensure that sensitive credentials are not exposed in public code repositories.
- Maintain separate test and production environments.

Credential hardcoding risks

While hardcoding credentials can seem like an option for the development process, it creates significant risks in your applications. If a malicious actor can gain access to your codebase, they can easily gain access to the credentials. The malicious actor can then control any resource the credentials are authorized to access.

For example, if a malicious actor finds your Amazon SP-API credentials, they can make API calls on your behalf without your permission or knowledge. It is possible that you might never know whether an attacker is using your credentials, since it is difficult to discern whether access is legitimate or illegitimate.

Rather than hardcoding credentials, solution providers must store credentials in a secure credential store, such as [AWS Secrets Manager](#). Your application can then call the credential store to retrieve necessary credentials only when needed.

Selling Partner API application credentials

For solution providers with [Selling Partner API](#) (SP-API) applications, SP-API relies on [Login with Amazon](#) (LWA) authorization codes, access tokens, and refresh tokens.

Solution providers must protect these credentials according to the requirements in the DPP.

Amazon's credential protections

Amazon continually scans the internet to detect if sensitive credentials are publicly available. When Amazon finds keys on the internet, the Information Security team rotates the keys to remove the risk and block malicious actors from taking control of the application. Amazon then informs the key owner.

Since the keys have been made publicly available, Amazon assumes that an unauthorized party has access to the keys, putting the Amazon SP-API application owners at risk. Unfortunately, rotating the keys means that the application owner cannot control the application, either.

While this process helps protect Amazon SP-API applications, it causes significant interruption for application owners and their customers. Typically, Amazon SP-API solution providers don't realize that Amazon has rotated their keys and they spend valuable time researching the root cause of their interruption. It can take days for a solution provider to realize that Amazon rotated the keys and contact Amazon via a support case.

Special Authorization Types

- > This is the only mechanism that Amazon has to protect your credentials. If another credential such as a decryption key were to become public, Amazon would not be able to disable or rotate the key. An attacker would have the authority to decrypt your sensitive information, without Amazon's or your solution provider's awareness.

INTEGRATION

Building Robust Amazon SP-API Applications

SP-API SDKs

>

Best practices for handling credentials

The following sections discuss best practices for handling credentials.

Layer protection

Solution providers must protect credentials in a secure credential store. A credential store protects the secrets that are needed to access applications, services, and IT resources. Using a credential store allows you to have a single repository of all secrets, allowing for easier management to protect, rotate, manage, and use secrets across applications, services, and other protected resources.

Since a single credential store might store all of your secrets, it is imperative that the credential store is protected. To protect your credential store:

- Limit access to only the people and resources that require access.
- Require Multi-Factor Authentication (MFA) for all user accounts.
- Encrypt the contents with strong encryption algorithms.
- Establish robust logging, monitoring, and alerting systems that enable rapid incident response.
- Implement a Key Management System (KMS) that covers the complete key lifecycle management.

Delete an Application From Your Developer Account

CALLING THE SP-API

Configuration Details

>

Call Structure

>

Development Tools

>

Performance Management

>

Control user access and passwords

Controlling user access is essential for preventing unauthorized credential exposure in any system. Solution providers must implement strict protocols around login credentials, avoiding the creation or use of generic, shared, or default accounts while preventing credential sharing between users. This starts with establishing a formal user access registration process that carefully assigns appropriate access rights for all user types and services, ensuring each person with computer access receives a unique ID.

Access management requires ongoing vigilance. To ensure that only required users can access information at any given time, solution providers must:

- Prevent employees and contractors from storing information on personal devices, as this creates uncontrolled access points that could bypass established security measures.
- Encrypt Amazon-provided API keys and limit their access strictly to required employees.
- Review access lists for both people and services every quarter.
- Disable employee access within 24 hours when they depart.

Account security is further strengthened through automated protection mechanisms. Solution providers must:

SOLUTION PROVIDER PORTAL

Manage Users in Solution Provider Portal

Manage Your Business And Contact information in Solution Provider Portal

Update Your Developer Profile in Solution Provider Portal

Unify Your Developer Accounts

Verify Your Identity in Solution Provider Portal

API Usage Metrics in Solution Provider Portal

TUTORIALS

Tutorial: Subscribe to the ORDER_CHANGE Notification

Tutorial: Test Selling Partner API Endpoints

Tutorial: Automate your SP-API Calls Using a C# SDK

Tutorial: Automate your SP-API Calls Using a Java SDK

Tutorial: Automate your SP-API Calls Using a JavaScript SDK for Node.js

Tutorial: Automate your SP-API Calls Using a Python SDK

Tutorial: Automate your SP-API Calls using a prebuilt C# SDK

Tutorial: Automate your SP-API Calls using prebuilt Java SDK

Tutorial: Automate your SP-API Calls using a prebuilt JavaScript SDK

Tutorial: Automate your SP-API Calls using prebuilt PHP SDK

Tutorial: Automate your SP-API calls using a prebuilt Python SDK

Tutorial: Authorize Multiple Vendor Central Accounts with a Single SP-API Application

Tutorial: Retrieve Merchant Shipping Templates

Tutorial: Grant the SP-API Permission to an Amazon SQS Queue

Tutorial: Retrieve and Pass a Purchase Order Number to a Carrier

TROUBLESHOOTING

Troubleshoot SP-API Errors

URL Encoding

Resolve Common HTTP and Authorization Error Codes

External Fulfillment Errors 

Listings Items API Issues Troubleshooting 

SELLING PARTNER APPSTORE

What is the Selling Partner Appstore?

List Your App on the Selling Partner Appstore

- Implement account lockout systems that can detect unusual usage patterns and login attempts.
- Automate account locking after 10 or fewer unsuccessful login attempts to prevent brute force attacks.
- Prevent the reuse of the last 10 passwords, ensuring users cannot revert to potentially compromised credentials.

Solution providers must ensure that passwords:

- Have a minimum length of 12 characters.
- Do not include any part of the user's name, including first name, last name, username, or email address.
- Include a mix of upper-case letters, lower-case letters, numbers, special characters, including minimum requirements for each character.
- Have a minimum age of one day.
- Expire after a maximum of 365 days.

Schedule and automate key rotation

A best practice is to assume that your credentials will be exposed eventually. Solution providers must mitigate this risk by establishing a credential rotation schedule. Scheduling key rotation reduces the window of time that a compromised credential can be used by unauthorized users.

Key rotation is usually prompted by specific actions, such as the key holder relinquishing their need-to-know role, or the length of time that a key has been in service. For best practices, solution providers must automatically rotate API keys and associated credentials every year.

Maintain private code repositories

A public repository exposes your code to the world, making it available to competitors and attackers alike. This presents a significant risk of exposing your company's normal operations. For this reason, Amazon recommends that in addition to following the recommendations in this guide, you also ensure that your repositories remain private.

Solution providers must implement additional measures to protect credentials and sensitive assets. Ensure that the credentials are not exposed in the repositories.

If you are a solution provider who uses GitHub as a primary code repository hosting service, you should refer to [Setting repository visibility](#) in GitHub's documentation to learn how to make your repositories private. You should also consider leveraging built-in features from these services including [automatic code scanning to detect credentials](#) and [encrypting secrets natively](#).

Guidance for Amazon Web Services (AWS) Customers

The following guidance is tailored to issues AWS customers face, however, other cloud provider customers and solution providers with an on-premise environment can also leverage equivalent services to achieve the same goal.

[Edit Your Appstore Listing](#)[Check Listing Status](#)[Appstore Ratings and Reviews](#)[Press Releases and Promotions](#)**SERVICE PROVIDER NETWORK**[List Your Service](#)**SECURITY AND COMPLIANCE**[SP-API Security and Compliance Overview](#)[Amazon Selling Partner API Guard Implementation Guide](#) >[VAT Calculation Service](#) >[Amazon Seller Data Access](#)[Best Practices](#) >[Key Security Control Guidance](#)[Logging and Monitoring for Amazon SP-API Applications](#)[Network Protection Guidance](#)[Protect Amazon API Applications with Data Encryption](#)[Protect Amazon API Applications with Data Retention and Recovery](#)[Protect Amazon SP-API Applications with Incident Response](#)[Safeguarding Sensitive Credentials](#)

Solution providers can take advantage of AWS IAM roles to reduce authentication and control process efforts. However, some services and applications require credentials with extended validity periods, in cases such as API keys, password databases, and SSH keys, which should not be hardcoded into application source code.

AWS Secrets Manager helps you manage, retrieve, and rotate database credentials, API keys, and other secrets throughout their lifecycles. Refer to the [AWS Secrets Manager documentation](#) for a basic scenario for using AWS Secrets Manager, tutorials, and guides. To avoid hardcoding credentials, solution providers can set up Secrets Manager for users and services to retrieve credentials using Secrets Manager API calls.

To increase effectiveness, it is recommended that you learn how to use AWS IAM roles for applications running on EC2. You should also learn how to securely provide credentials to AWS Lambda functions with Secrets Manager.

Key takeaways and next steps

- Eliminate hardcoded credentials immediately by auditing all source code, documentation, and configuration files and remove embedded API keys, passwords, and secret keys that create security vulnerabilities.
- Deploy secure credential infrastructure, including encrypted credential stores, complete KMS lifecycle management, and restricted API key access.
- Implement mandatory access controls with formal user registration processes, MFA for all accounts, account lockout after 10 failed attempts, and password history preventing reuse of last 10 passwords.
- Establish automated security maintenance, including annual API key rotation, quarterly access reviews, and 24-hour access removal for terminated employees.
- Consult the latest version of the [Data Protection Policy](#) and [Acceptable Use Policy](#) for the most up-to-date compliance requirements.

For additional information, refer to:

- [Best Practices for Security, Identity & Compliance](#)
- [Setting repository visibility](#)
- [Store, Distribute, and Rotate Credentials Securely](#)
- [Top 10 security items to improve in your AWS account](#)

 Updated 18 days ago

← [Protect Amazon SP-API Applications with Incident Response](#)

[Vulnerability Management](#) →

Did this page help you?  Yes  No

A+ CONTENT API

A+ Content API >

A+ Content Examples

AMAZON WAREHOUSING AND DISTRIBUTION API

Amazon Warehousing and Distribution API >

Amazon Warehousing and Distribution API v2024-05-09
Reference

APP INTEGRATIONS API

App Integrations API >

APPLICATION MANAGEMENT API

Application Management API >

CATALOG ITEMS API

Catalog Items API >

Catalog Items API Rate Limits
Catalog Items API v2022-04-01 Reference

CUSTOMER FEEDBACK API

Customer Feedback API >

Customer Feedback API Rate Limits

DATA KIOSK

Data Kiosk API >

Data Kiosk Schema Explorer User Guide

Data Kiosk Query Processing Finished Notification

Building Data Kiosk workflows guide

Vendor Analytics Dataset Use Case Guide

Schedule Data Kiosk Queries

DELIVERY BY AMAZON API

Delivery by Amazon API >

EASY SHIP API

Easy Ship API >

EXTERNAL FULFILLMENT

External Fulfillment Inventory API >

External Fulfillment Returns API >

External Fulfillment Shipping API >

External Fulfillment Errors

External Fulfillment APIs Rate Limits

FULFILLMENT BY AMAZON (FBA)

FBA Inventory API >

FBA Inventory Dynamic Sandbox Guide

FEEDS API

Feeds API >

Feed Type Values >

Feeds JSON Schemas ↗

Feeds API Best Practices

Feeds API FAQ

Feeds API Rate Limits

FINANCES

Finances API >

Transfers API >

FULFILLMENT INBOUND API

Fulfillment Inbound API >

Fulfillment Inbound API FAQ

Migrating Fulfillment Inbound workflows

Fulfillment Inbound API Rate Limits

FULFILLMENT OUTBOUND API

Fulfillment Outbound Dynamic Sandbox Guide

Fulfillment Outbound API >

Fulfillment Outbound API Rate Limits

MCF Best Practices

INVOICING

Invoices API >

Invoices API FAQ

LISTINGS

Listings Items API >

Listings Items API Rate Limits

Listings Restrictions API >

Listings Restrictions API Rate Limits

Manage Product Listings with the Selling Partner API

Building Listings Management Workflows Guide

Understanding Amazon listing status and seller-fulfilled inventory management

Listings Management Workflow Migration >

Manage Amazon Haul, Advanced Multiple-Offer, and Multiple-Fulfillment Use Cases

Listings APIs FAQ

Product Type Definitions API >

Product Type Definitions API Rate Limits

MERCHANT FULFILLMENT API

Merchant Fulfillment API >

MESSAGING API

Messaging API >

NOTIFICATIONS API

Notifications API >

Notification Type Values

Tutorial: Subscribe to the ORDER_CHANGE Notification ↗

ORDERS API

Orders API >

Tutorial: Retrieve and Pass a Purchase Order Number to a Carrier ↗

Orders API Migration Guide

Orders API Rate Limits

PRODUCT FEES API

Product Fees API >

PRODUCT PRICING API

Product Pricing API >

Product Pricing API and Notifications FAQ

Price Adjustment Automation Workflows Guide
Manage automated pricing rules with SP-API

REPLENISHMENT API

Replenishment API >

REPORTS API

Reports API >

Report Type Values >

Reports JSON Schemas ↗

Reports API FAQ

SALES API

Sales API >

SELLER WALLET API

Seller Wallet API >

Seller Wallet API Rate Limits

SELLERS API

Sellers API >

SERVICES API

Services API >

SHIPMENT INVOICING API

Shipment Invoicing API >

SHIPPING API

Shipping API v2 Resources ↗

Shipping API v1 Reference

SOLICITATIONS API

Solicitations API >

SUPPLY SOURCES API

Supply Sources API >

Multi-Location Inventory Integration Guide

Supply Sources API Rate Limits

TOKENS API

Tokens API >

UPLOADS API

Uploads API >

VEHICLES API

Vehicles API >

Vehicles API Rate Limits

VENDOR DIRECT FULFILLMENT APIS

Vendor Direct Fulfillment Dynamic Sandbox Guide

Vendor Direct Fulfillment Workflow Guide

SP-API Bill-to-Party Addresses

VENDOR DIRECT FULFILLMENT TRANSACTION STATUS API

Vendor Direct Fulfillment Transaction Status API >

VENDOR DIRECT FULFILLMENT INVENTORY API

Vendor Direct Fulfillment Inventory API >

VENDOR DIRECT FULFILLMENT ORDERS API

Vendor Direct Fulfillment Orders API >

VENDOR DIRECT FULFILLMENT SHIPPING API

Vendor Direct Fulfillment Shipping API >

VENDOR DIRECT FULFILLMENT PAYMENTS API

Vendor Direct Fulfillment Payments API >

VENDOR RETAIL PROCUREMENT INVOICES API

Vendor Retail Procurement Invoices API >

VENDOR RETAIL PROCUREMENT ORDERS API

Vendor Retail Procurement Orders API >

VENDOR RETAIL PROCUREMENT SHIPMENTS API

Vendor Retail Procurement Shipments API >

VENDOR RETAIL PROCUREMENT TRANSACTION STATUS API

Vendor Retail Procurement Transaction Status API >